

Стандартная библиотека шаблонов

Векторы

Стандартная библиотека шаблонов (STL)

- **STL (Standart Template Library)**, стандартная библиотека шаблонов — набор согласованных обобщённых алгоритмов, контейнеров, средств доступа к их содержимому и различных вспомогательных функций в C++.
- Разработана Александром Степановым и Менг Ли, работающими в **Hewlett-Packard Lab**, им помогал Д. Л. Муссер из Ренсселэровского политехнического института.
- Стандарт языка не называет её «STL», так как эта библиотека стала неотъемлемой частью языка, однако многие люди до сих пор используют это название, чтобы отличать её от остальной части стандартной библиотеки (поток ввода-вывода (iostream), подраздел Си и др.).

Компоненты библиотеки:

1. Контейнер (англ. container) — хранение набора объектов в памяти.
2. Итератор (англ. iterator) — обеспечение средств доступа к содержимому контейнера.
3. Алгоритм (англ. algorithm) — определение вычислительной процедуры.
4. Адаптер (англ. adaptor) — адаптация компонентов для обеспечения различного интерфейса.
5. Функциональный объект (англ. functor) — сокрытие функции в объекте для использования другими компонентами.

Контейнеры

- Контейнеры (containers) - это объекты, предназначенные для хранения других элементов. Например, вектор, линейный список, множество и т.п.
- У каждого контейнера имеется определенный для него распределитель памяти (**Allocator**), который управляет процессом выделения памяти для контейнера.
- По умолчанию распределителем памяти является объект класса **Allocator**. Можно определить собственный распределитель.
- В каждом классе-контейнере определен набор функций для работы с ними.

Алгоритмы

- Алгоритмы (**algorithms**) выполняют операции над содержимым контейнера.
- Существуют алгоритмы для инициализации, сортировки, поиска, замены содержимого контейнеров.

Итераторы

- Итераторы (**iterators**) - это объекты, которые по отношению к контейнеру играют роль указателей.
- Они позволяют получить доступ к содержимому контейнера примерно так же, как указатели используются для доступа к элементам массива.
- С итераторами можно работать так же, как с указателями. К ним можно применить операции *, инкремента, декремента.
- Типом итератора объявляется тип **iterator**, который определен в различных контейнерах.

Существует пять типов итераторов:

1. Итераторы ввода (**input iterator**) поддерживают операции равенства, разыменования и инкремента.

`==, !=, *i, ++i, i++, *i++`

Специальным случаем итератора ввода является **istream_iterator**.

2. Итераторы вывода (**output iterator**) поддерживают операции разыменования, допустимые только с левой стороны присваивания, и инкремента.

`++i, i++, *i = t, *i++ = t`

Специальным случаем итератора вывода является **ostream_iterator**.

3. Однонаправленные итераторы (**forward iterator**) поддерживают все операции итераторов ввода/вывода и, кроме того, позволяют без ограничения применять присваивание.

`==, !=, =, *i, ++i, i++, *i`

типы итераторов:

4. Двухнаправленные итераторы (**bidirectional iterator**) обладают всеми свойствами forward-итераторов, а также имеют дополнительную операцию декремента (`--i`, `i--`, `*i--`), что позволяет им проходить контейнер в обоих направлениях.

5. Итераторы произвольного доступа (**random access iterator**) обладают всеми свойствами bidirectional-итераторов, а также поддерживают операции сравнения и адресной арифметики, то есть непосредственный доступ по индексу.

```
i += n, i + n, i -= n, i - n, i1 - i2, i[n],  
i1 < i2, i1 <= i2, i1 > i2, i1 >= i2
```


Векторы в C++

- **Вектор** — это структура данных, которая уже является моделью динамического массива.
- Для создания динамического массива вручную нужно пользоваться конструктором **new** и указателями.
В случае с векторами всего этого делать не нужно.
- По стандарту пользоваться динамическим массивом через конструктор **new** — не очень правильно, т. к. высока угроза, что будут происходить различные утечки памяти.

Создание вектора

- Необходимо подключить заголовочный файл `<vector>`:
- **`#include <vector>`**
- `vector<тип элементов> имя (размер) ;`
- Такой способ объявления вектора не просто выделяет память, **но и инициализирует все элементы вектора нулями.**
- `vector<тип элементов> имя; // динамический`
- `vector<тип элементов> имя (элементы через запятую) ;`

Пример создания вектора

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v1(10); //фиксированный размер
    vector<float> v2;   //неопределенный размер
    v2.reserve(10); // зарезервировали память под 10
    //элементов типа float, но нет начальной инициализации
    vector<int> v3(10, 0, 7); //инициализация
        return 0;
}
```

- Согласно стандарту C++, любой контейнер должен содержать следующие методы:
- **begin()** - возвращает итератор на первый элемент вектора.
- **end()** - возвращает итератор на место после последнего элемента вектора.
- **size()** - возвращает количество элементов в векторе.
- **max_size()** - возвращает максимально возможное количество элементов в векторе.
- **empty()** - возвращает true, если вектор пуст.
- **swap()** - обменивает содержимое двух векторов.

Доступ к элементам вектора

- **at(size_type n)** - доступ к элементу с индексом n. При некорректном значении n возникает ошибка!
- **back()** - доступ к последнему элементу;
- **front()** - доступ к первому элементу.
- Имя вектора[n] - доступ к элементу с индексом n. **Не генерирует исключение!**

Методы класса vector

- **clear()** — удалить все элементы вектора
 - **pop_back()** — удалить последний элемент
 - **erase(iterator position)** - удаляет указанный элемент и возвращает итератор на следующий элемент или end();
 - **erase(iterator first, iterator last)** - удаляет элементы из диапазона [first, last).
 - **insert(iterator position, const T& x)** - вставляет элемент со значением x в указанную позицию;
 - **insert(iterator position, InputIterator first, InputIterator last)** - вставляет в указанную позицию копии элементов диапазона [first, last).
- Без параметров

Перегруженные операции для вектора

- `[]` - операция индексации, **не генерирует исключение при выходе за границы массива**;
- `=` - назначить одному вектору копии элементов второго (`v1 = v2`);
- `==` - определить равны ли два вектора (`v1 == v2`);
- `!=` - определить не равны ли два вектора (`v1 != v2`);
- `<` - меньше ли один вектор чем другой (`v1 < v2`);
- `>` - больше ли один вектор чем другой (`v1 > v2`);
- `<=` - меньше или равен один вектор по отношению к другому (`v1 <= v2`);
- `>=` - больше или равен один вектор по отношению к другому (`v1 >= v2`).

Пример использования итератора

```
int A[3]={100,200,300}; //массив для инициализации вектора  
vector<int> v(A,A+3);  
//объявляем вектор и записываем в него три элемента из массива  
vector<int>::iterator it; //объявляем итератор it  
  
for (it = v.begin(); it!=v.end();it++)  
cout<<*it<<" ";  
// итератор используется для доступа к элементам вектора
```


Разница между итератором и указателем

- Главное предназначение итераторов заключается в предоставлении возможности пользователю обращаться к любому элементу контейнера при сокрытии внутренней структуры контейнера от пользователя.
- Это позволяет контейнеру хранить элементы любым способом при допустимости работы пользователя с ним как с простой последовательностью или списком.
- Указатель может указывать на абсолютно любой участок памяти, а итератор— строго на начало **конкретного элемента**, по сути, превращаясь в индекс.
- И второе, отдельный тип `iterator` в первую очередь нужен самой STL, а именно, компилятору, чтоб правильно выбрать прототип шаблонной функции.

Сравнение векторов с массивами

Вектор

- `vector<int> myVector(10);`
- Инициализация по умолчанию {0}
- Наличие итераторов
- Возможность изменения размера

Одномерный массив

- `int myMas[10];`
- При отсутствии инициализации содержит «мусор»
- Отсутствует контроль выхода за границы массива
- Создается статически